Lab Assignment: Command-Line To-Do List Application

Objective

Your goal is to build a fully functional to-do list application that runs in the command line. The application will allow a user to add, view, complete, and delete tasks. All tasks will be saved to a local file so that they persist even after the program is closed and reopened.

This assignment will test your understanding of functions, lists, dictionaries, conditional logic, data validation, and file I/O.

Core Requirements

1. Data Structure

Your primary data structure for holding the to-do items will be a **list of dictionaries**. Each dictionary within the list represents a single task and must contain at least two key-value pairs:

- A key for the task's description (e.g., `'description': 'Buy milk'`).
- A key to track its completion status (e.g., `'completed': False`).

2. File Persistence

- The application must load tasks from a file (e.g., `tasks.txt`) when it starts.
- If the `tasks.txt` file does not exist, the program should start with an empty to-do list.
- Whenever a change is made (a task is added, completed, or deleted), the entire list of tasks must be saved back to the `tasks.txt` file.

3. Main Application Loop & User Menu

- The program should continuously run, displaying a menu of options to the user until they explicitly choose to exit.
- The menu should present the following options:

1. View all tasks
2. Add a new task
3. Mark a task as complete
4. Delete a task
5. Exit

4. Functionality

Create a separate, well-named function for each of the main features:

- **`add_task()`**: Prompts the user for a task description. It should then create a new task dictionary and add it to your main list of tasks.
- **`view_tasks()`**: Prints all tasks to the console. Each task should be numbered and clearly show its completion status. For example, use `[x]` for completed tasks and `[ ]` for incomplete ones. If no tasks exist, it should print a friendly message.
- **`mark_task_complete()`**: Asks the user for the number of the task they wish to complete. You must update the status of that specific task in your list.
- **`delete_task()`**: Asks the user for the number of the task they want to delete and removes it from your list.
- **`load_tasks()`** and **`save_tasks()`**: Functions to handle reading from and writing to your `tasks.txt` file.

5. Input and Data Validation

- Your application must gracefully handle incorrect user input.
- When a user is asked to select a task by its number (for completing or deleting), you must validate their input.
- Check if the input is a valid number using methods like `.isdigit()` or a `try-except` block.
- Check if the number corresponds to an actual task in the list.
- If the input is invalid, print a clear error message and allow the user to try again.

Expected Output and Behavior

Here is a sample walkthrough of how your finished application should behave.

On first run (when `tasks.txt` does not exist):

```
CODE
--- To-Do List Menu ---
1. View all tasks
2. Add a new task
3. Mark a task as complete
4. Delete a task
5. Exit
Enter your choice: 1
Your to-do list is empty.
```

Adding new tasks:

```
CODE
Enter your choice: 2
Enter the task description: Finish the CLI project
Task 'Finish the CLI project' added.

Enter your choice: 2
Enter the task description: Water the plants
Task 'Water the plants' added.
```

Viewing the list of tasks:

```
CODE
Enter your choice: 1
--- YOUR TASKS ---
1. [ ] Finish the CLI project
2. [ ] Water the plants
--------------------
```

Marking a task as complete:

```
CODE
Enter your choice: 3
Enter the number of the task to mark as complete: 1
Task 'Finish the CLI project' marked as complete.
```

Viewing the list again:

```
CODE
Enter your choice: 1
--- YOUR TASKS ---
1. [x] Finish the CLI project
2. [ ] Water the plants
--------------------
```

Handling invalid input:

```
CODE
Enter your choice: 3
Enter the number of the task to mark as complete: 99
Error: Task #99 not found.

Enter your choice: hello
Error: Invalid input. Please enter a number from 1 to 5.
```

Deleting a task:

```
CODE
Enter your choice: 4
Enter the number of the task to delete: 2
Task 'Water the plants' has been deleted.
```

Exiting the program:

```
CODE
```

```
Enter your choice: 5
Goodbye!
```

(Now, if you were to restart the program, it would load the remaining task: "1. [x] Finish the CLI project")